I will now generate the consolidated, final draft of the **HandoffHQ Architecture & Implementation Blueprint** in a single Markdown file (HandoffHQ_Final_Architecture_Blueprint.md). It merges all confirmed product decisions, the 6-gate deterministic Readiness Engine, the hardened two-state persistence model (SUPABASE_CONNECTED vs. LOCAL_DEMO), the Tailwind CSS v4 setup, and the self-contained coding-agent handoff specification.

HandoffHQ Final Architecture Blueprint
Export

HandoffHQ — Final Architecture & Implementation Blueprint

**Document Status:** Final Consolidated Draft (Includes Technical Review & Security Amendments)
**Event & Track:** Kalvium Hackathon — Track 3 (Notion / CRM / Agency Operations)

Edit directly or with Gemini
Click anywhere to type and edit directly, or select text to prompt Gemini for changes.
**Execution Profile:** Solo Beginner Developer | $< 24$ Hours Available | Zero-Cost Cloud Stack

# 0. Executive Summary, Confirmed Baseline & Explicit Assumptions

## 0.1 Confirmed Product Concept

- **Product Definition: HandoffHQ** is a **Sales-to-Delivery Handover Command Center** for service agencies. It manages the transition from a newly won client deal to an execution-ready project for the delivery team.
- **Core Problem:** When an agency wins a client, critical operational details—agreed scope, promised deliverables, deadlines, deal budget, client dependencies/credentials, project owners, and signed documents—are scattered across chats, emails, and sales reps' memories. Delivery teams start work without full context, causing rework, missed commitments, scope disputes, and lost client trust.
- **Winning Differentiator — The Readiness Engine:** Unlike a generic CRM, Kanban

board, or Notion clone, HandoffHQ enforces an explainable, deterministic **Readiness Engine**. It evaluates 6 mandatory handover rules in real time, displays completion percentage and readiness status (Draft, Blocked, Ready for Delivery, Handed Over), identifies each missing requirement, explains *why* it blocks the transition, tells the user *what action* resolves it, blocks premature handoffs, and records an append-only **Handover Activity History**.

- **The 60-Second Judge Demo:** Centered on a realistic sample project called **"Acme Website Redesign" ($45,000)** that starts in Blocked status (3/6 gates passed). The presenter resolves the 3 explicit blockers live, watches the Readiness Engine unlock the **"Mark Ready for Delivery"** transition, switches persona to the receiving Delivery Lead, accepts the handover into **Handed Over** status, and showcases the persisted activity history.

## 0.2 Explicit Technical & Scope Assumptions

To protect the $< 24$-hour solo-developer deadline, the following assumptions are locked:

1. **Assumption A-01 (JavaScript .jsx over TypeScript .tsx):** The frontend uses **React 18 + Vite with plain JavaScript (.jsx)** rather than TypeScript. *Why:* Debugging TypeScript generics on Supabase return shapes can cost 3–4 hours without improving the judge-facing product.
2. **Assumption A-02 (Top-Bar Demo Role Switcher over Login Screens):** Instead of email/password authentication walls that slow down judges during a 60-second review, the top navbar provides a **Demo Role Switcher** to toggle between 3 pre-seeded personas (Sales Rep, Delivery Lead, Agency Ops Manager).
3. **Assumption A-03 (Document URLs over Binary File Uploads):** Supporting documents (Signed SOW, Figma Specs, Brand Kits) are stored as validated https:// URLs rather than binary file uploads, avoiding S3 bucket and multipart upload complexity.
4. **Assumption A-04 (PostgreSQL JSONB Checklists):** Child collections (deliverables, dependencies, document_links, activity_logs) are stored as structured JSONB arrays inside a single handovers table in Supabase PostgreSQL. This turns multi-table joins and transactions into a single atomic row read/write.

# Deliverable A: Product Scope, MVP Boundaries & Readiness Rules

## A.1 Strict MVP Boundaries

- **Must-Have (In Scope for MVP):**
  - **Command Center Dashboard:** 4 KPI summary cards (Active Pipeline Value, Blocked Count, Ready for Delivery Count, Handed Over Count), status filter pills, and handover project cards with readiness progress bars.
  - **Pre-Seeded Agency Portfolio:** 4 realistic handovers headlined by **"Acme Website Redesign"** (starts Blocked at 50% / 3 of 6 gates passed).

    - **Split-Screen Handover Workspace:** Left column for editing project overview, scope summary, deliverables, mandatory/optional client dependencies, supporting document links, and delivery owner; right sticky column for the live **Readiness Engine Panel**.
    - **Deterministic 6-Gate Readiness Engine:** Real-time evaluation of Rules R-01 through R-06, blocker explanations, remediation instructions, and strict role-based transition locks.
    - **Append-Only Handover Activity History:** Chronological log recording actor name, role badge, action description, and timestamp on every save and status change.
    - **1-Click "Reset Demo Data" Button:** Idempotently restores all 4 demo projects to their initial starting states in under 2 seconds between judge visits.
- **Cut from Primary Path (Out of Scope to Protect Deadline):**
    - Email/password or OAuth login walls.
    - LLM/AI readiness guesses (explicitly replaced by deterministic rules).
    - External Slack/Email/CRM webhooks.
    - Binary file storage buckets.

## A.2 The 6 Mandatory Readiness Engine Rules

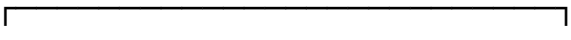
Every edit and every status transition synchronously executes the pure function evaluateReadiness(handover). All 6 gates (6/6 = 100%) must pass before a handover can transition to Ready for Delivery or Handed Over:

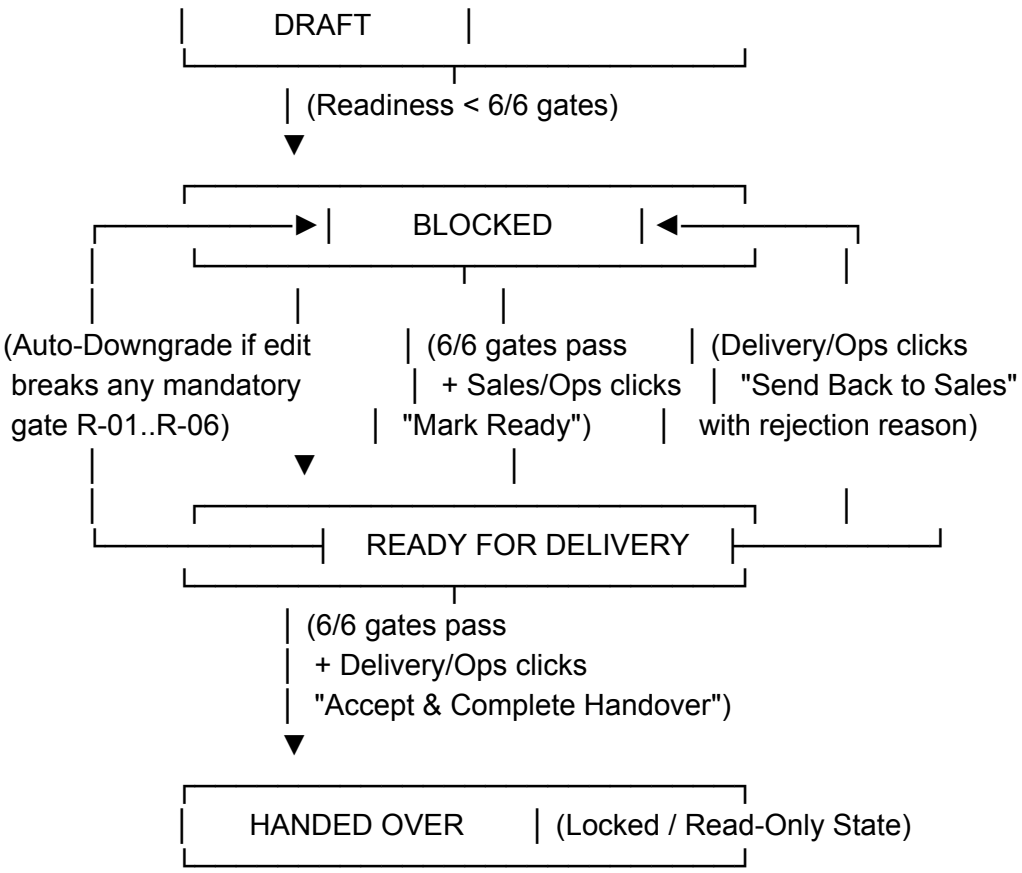
| Gate ID | Rule Title | Exact Deterministic Pass Condition | "Why This Blocks Handover" Explanation | Action Needed to Resolve |
|---|---|---|---|---|
| **R-01** | **Commercial & Timeline Baseline** | typeof h.client_name === 'string' && h.client_name.trim().length >= 2 && Number.isFinite(Number(h.deal_value)) && Number(h.deal_value) > 0 && /^\d{4}-\d{2}-\d{2}$/.test(h.target_launch_date | Delivery cannot schedule capacity or track budget without a confirmed client, positive deal value, and target launch date. | Enter Client Name, Deal Value ($ > 0), and Target Launch Date in Overview. |

| | | | | |
|---|---|---|---|---|
| | | \|\| '') && !Number.isNaN(Date.parse(h.target_launch_date)) | | |
| **R-02** | **Detailed Scope Summary** | typeof h.scope_summary === 'string' && h.scope_summary.trim().length >= 30 | Vague or one-line scope notes cause immediate scope creep and client disputes during kickoff. | Write a clear Scope Summary of at least 30 non-whitespace characters defining project boundaries. |
| **R-03** | **Agreed Deliverables** | Array.isArray(h.deliverables) && h.deliverables.length >= 2 && h.deliverables.every(d => d && typeof d.title === 'string' && d.title.trim().length >= 3 && /^\d{4}-\d{2}-\d{2}$/.test(d.due_date \|\| '')) | Delivery leads cannot build a sprint schedule without at least 2 concrete deliverables and milestone dates. | Add at least 2 deliverables with valid titles and target dates. |
| **R-04** | **Mandatory Client Dependencies** | Array.isArray(h.dependencies) && h.dependencies.filter(d => d && d.is_mandatory === true).length >= 1 && | Starting engineering or design work without mandatory client credentials or environment access stalls the team | Ensure at least 1 mandatory dependency exists and check off all mandatory items as resolved. |

| | | | | |
|---|---|---|---|---|
| | | h.dependencies.filter(d => d && d.is_mandatory === true).every(d => d.is_resolved === true) | mid-sprint. | |
| **R-05** | **Supporting Documents Linked** | Array.isArray(h.document_links) && h.document_links.length >= 1 && h.document_links.every(doc => doc && doc.label?.trim().length >= 2 && isValidHttpUrl(doc.url)) | Verbal sales promises are unenforceable without a linked SOW, signed proposal, or technical specification. | Attach at least 1 valid http:// or https:// link to supporting documentation. |
| **R-06** | **Assigned Delivery Owner** | Boolean(h.delivery_owner_id && h.delivery_owner_id.trim() && h.delivery_owner_name && h.delivery_owner_name.trim()) | An unassigned handover sits in limbo with zero accountability for client onboarding and kickoff. | Select and assign a receiving Delivery Lead from the Delivery Owner dropdown. |

## A.3 Valid Status Transitions & Role Enforcement Matrix






| Action / Transition | Allowed From Status | Readiness Requirement | Permitted Roles | Edge-Case Behavior |
|---|---|---|---|---|
| **Save Field / Checklist Edits** | Draft, Blocked, Ready for Delivery | Any (0/6 to 6/6) | sales, delivery, ops | **Auto-Downgrade Guard:** If status is Ready for Delivery and an edit causes passedCount < 6, status automatically reverts to Blocked and logs the downgrade reason. |

| | | | | |
|---|---|---|---|---|
| **Mark Ready for Delivery** | Draft, Blocked | **Must be 6/6 (100%)** | sales, ops | Service re-runs evaluateReadiness(). Throws error if <6/6. Disabled for delivery role with helper prompt. |
| **Accept & Complete Handover** | Ready for Delivery | **Must be 6/6 (100%)** | delivery, ops | Service re-runs evaluateReadiness(). Sets status to Handed Over and locks workspace into read-only mode. |
| **Send Back to Sales** | Ready for Delivery | Any | delivery, ops | Requires a rejection reason ($\geq 5$ chars). Sets status to Blocked and logs the reason. |
| **Edit a Handed Over Deal** | Handed Over | Locked | **None** | Service rejects edits once Handed Over. Only resetDemoData() can reset the record. |

# Deliverable B: User Personas & Core User Journeys

## B.1 Pre-Seeded Demo Personas (src/constants/demoUsers.js)

1. **Aarav Mehta — Account Executive (role: 'sales')**
    - **Responsibility:** Creates handovers after closing a deal, resolves readiness blockers, and submits the package when all 6 gates pass.
2. **Priya Nair — Delivery Lead (role: 'delivery')**
    - **Responsibility:** Reviews incoming Ready for Delivery handovers, verifies dependencies and scope, and either accepts (Handed Over) or sends back to Sales with a rejection note.
3. **Vikram Desai — Head of Agency Operations (role: 'ops')**
    - **Responsibility:** Monitors agency-wide handover quality, pipeline value stuck in Blocked status, and activity logs; holds override permissions for both transitions during demos.

## B.2 The 60-Second Judge Demo Script ("Acme Website Redesign")

1. **Step 1 — Spot the Problem (0:00–0:15):** Start on the Command Center Dashboard as **Aarav Mehta (Sales)**. Point out **"Acme Website Redesign ($45,000)"** marked **Blocked (3/6 Gates — 50%)**. Click to open its workspace.
2. **Step 2 — Show Explainable Blockers (0:15–0:30):** Point to the sticky **Readiness Engine Panel** on the right. Show why **"Mark Ready for Delivery"** is locked:
    - **R-02 Failed:** Scope summary is only 13 characters (*"Redesign site"*).
    - **R-04 Failed:** Mandatory dependency *"Production DNS & Webflow Workspace Admin Access"* is unresolved.
    - **R-06 Failed:** No receiving Delivery Owner is assigned.
3. **Step 3 — Resolve Blockers Live (0:30–0:45):**
    - Click the **"✨ Insert Full Scope Summary"** helper button (populates a 145-character scope summary → **R-02 turns green**).
    - Check the box for *"Production DNS & Webflow Workspace Admin Access"* (→ **R-04 turns green**).
    - Select **"Priya Nair (Delivery Lead)"** in the Delivery Owner dropdown (→ **R-06 turns green**).
    - The Readiness Gauge hits **100% (6/6 Gates Passed)** and unlocks **"🚀 Mark Ready for Delivery"**. Click it; status changes to **Ready for Delivery**.
4. **Step 4 — Role Handoff & Acceptance (0:45–1:00):** Switch the top-bar persona pill to **Priya Nair (Delivery Lead)**. Click **"✅ Accept & Complete Handover"**. Status locks to **Handed Over**, inputs become read-only, and the **Handover Activity History** displays every timestamped transition.

# Deliverable C: Prioritized Feature List & Acceptance Criteria

## Must-Have Features (P0 — Required for Demo)

| ID | Feature Name | User Story | Testable Acceptance Criteria |
|---|---|---|---|
| **F-01** | **Command Center Dashboard** | As any persona, I want a dashboard of all handovers, KPI totals, and readiness progress bars. | 1. Shows 4 KPI cards: Total Pipeline ($), Blocked, Ready for Delivery, Handed Over.<br><br>2. Filter tabs (All, Blocked, Ready for Delivery, Handed Over) filter the grid.<br><br>3. Each card shows client, project title, value, owner, status pill, and X/6 Gates progress bar. |
| **F-02** | **Deterministic Readiness Engine Panel** | As a Sales Rep, I want real-time visibility into missing requirements, why they block handoff, and how to fix them. | 1. Evaluates Rules R-01 to R-06 on every change.<br><br>2. Displays X/6 Mandatory Gates Passed and 0–100% bar.<br><br>3. Failed rules display in rose/amber with Why it blocks and Action needed text.<br><br>4. Passed rules display in emerald green. |
| **F-03** | **Structured Handover** | As a Sales Rep, I want to update | 1. Edits overview fields and includes |

| | | | |
|---|---|---|---|
| | **Workspace Editor** | scope, deliverables, dependencies, document links, and owner in one screen. | a 1-click "✨ Insert Full Scope Summary" helper.<br><br>2. Adds/removes deliverables with dates.<br><br>3. Toggles is_resolved on dependencies and adds new mandatory/optional items.<br><br>4. Validates document links for http:// or https:// protocol. |
| **F-04** | **Role-Gated State Machine** | As Delivery/Sales, I want transitions enforced by role and readiness score. | 1. "Mark Ready for Delivery" requires 6/6 gates + sales/ops role.<br><br>2. "Accept & Complete Handover" and "Send Back to Sales" require Ready for Delivery status + delivery/ops role.<br><br>3. Breaking a gate while Ready for Delivery auto-downgrades status to Blocked.<br><br>4. Handed Over locks all inputs. |

| | | | |
|---|---|---|---|
| **F-05** | **Append-Only Activity History** | As Ops, I want a chronological log of who changed what and when. | 1. Appends { id, actor_name, actor_role, action, timestamp } on saves and status changes.<br><br>2. Renders in reverse chronological order with role badges. |
| **F-06** | **Two-State Persistence & 1-Click Demo Reset** | As a presenter, I want reliable persistence and a 1-click reset between judge pitches. | 1. Displays active storage badge (Supabase Connected or Local Demo Mode).<br><br>2. Clicking "Reset Demo Data" restores all 4 fixed-UUID seed projects in <2s. |

### Should-Have Features (P1 — Build Only After P0 is Deployed)

- **F-07: + New Handover Creation Modal:** Pre-populates a new handover with standard agency mandatory dependencies. *(Deferred to P1 because the 60-second judge demo uses the pre-seeded "Acme Website Redesign" project).*

### Future Roadmap (P2 — Post-Hackathon)

- HubSpot / Pipedrive Deal Won webhook ingestion.
- Slack notifications to #delivery-handoffs.
- Automated Notion workspace template cloning on handover acceptance.

## Deliverable D: Recommended Tech Stack, Setup & Verified Free Tiers

| Layer | Technology Choice | Plain-Language Rationale | Verified Free Tier & Cost ($0.00) |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **Frontend** | **React 18+ & Vite (Plain JavaScript .jsx)** | Matches your existing JS/React/JSX skillset; instant HMR dev server; zero TypeScript compiler roadblocks. | **100% Free & Open Source** (MIT). |
| **Styling & Icons** | **Tailwind CSS v4 (@tailwindcss/vite ) + lucide-react** | Official 2026 Vite integration requires zero PostCSS or tailwind.config.js boilerplate—just @import "tailwindcss"; in index.css. | **100% Free & Open Source** (MIT). |
| **Database & API** | **Supabase (Managed PostgreSQL + @supabase/supabase-js)** | Eliminates building and hosting a separate Express server. React communicates directly with PostgreSQL via Supabase's auto-generated REST API. | **Verified Free Plan ($0/mo, no credit card):** 2 active free projects, 500 MB database storage, 5 GB egress, 50,000 MAUs. *(Free projects pause only after 7 days of inactivity).* |
| **Offline Fail-Safe** | **Explicit Two-State Service (SUPABASE_CONNECTED vs LOCAL_DEMO)** | Guarantees the demo works out-of-the-box before Supabase is configured and survives venue Wi-Fi drops without split-brain data bugs. | **100% Free** (Browser localStorage API). |
| **Hosting** | **Vercel (Hobby** | Zero-config GitHub deployment for Vite | **Verified Free Plan ($0/mo, no credit** |

| | | | |
|---|---|---|---|
| | **Plan)** | React apps with automatic HTTPS in ~45 seconds. | **card):** 100 GB bandwidth/month, 1M edge requests, 100 deploys/day for personal/hackathon use. |

# Deliverable E: Architecture Diagram & Component Responsibilities

Code snippet
flowchart TB
  subgraph Browser["Browser Client (React + Vite + Tailwind CSS v4)"]
    Nav["Navbar.jsx\n(Persona Switcher: Sales / Delivery / Ops\n+ Storage Mode Pill + 1-Click Demo Reset)"]
    Dash["DashboardView.jsx\n(4 KPI Cards, Status Filters, Project Cards Grid)"]
    Work["HandoverWorkspace.jsx\n(Left Column: Overview, Scope, Deliverables,\nDependencies, Docs, Activity History)"]
    Panel["ReadinessPanel.jsx\n(Right Sticky Column: Score Gauge, 6 Gate Cards,\nBlocker Explanations, Role-Gated Action Buttons)"]
    Engine["readinessEngine.js\n(Pure Deterministic Evaluator: Rules R-01 to R-06)"]
    Service["handoverService.js\n(Explicit Two-State Data Access & Transition Guards)"]
  end

  subgraph Storage["Persistence Layer (Explicit Mode Lock)"]
    Supa["SUPABASE_CONNECTED Mode\n(Supabase Cloud PostgreSQL + Scoped RLS)"]
    Local["LOCAL_DEMO Mode\n(Browser localStorage 'handoffhq_v1_handovers')"]
  end

  Nav --> Dash
  Nav --> Work
  Dash --> Service
  Work --> Engine
  Work --> Panel
  Panel --> Engine
  Work --> Service
  Panel --> Service
  Service -->|Mode = SUPABASE_CONNECTED| Supa

Service -->|Mode = LOCAL_DEMO| Local

# Deliverable F: Database Schema, Constraints & Security (supabase/schema.sql)

SQL

```
-- HandoffHQ Supabase PostgreSQL Schema
CREATE TABLE IF NOT EXISTS public.handovers (
    id UUID PRIMARY KEY,
    project_name TEXT NOT NULL CHECK (char_length(trim(project_name)) BETWEEN 1 AND
120),
    client_name TEXT NOT NULL CHECK (char_length(trim(client_name)) BETWEEN 1 AND
120),
    deal_value NUMERIC(12, 2) NOT NULL DEFAULT 0 CHECK (deal_value >= 0),
    target_launch_date DATE NULL,

    -- Strict state machine constraint
    status TEXT NOT NULL DEFAULT 'Blocked'
        CHECK (status IN ('Draft', 'Blocked', 'Ready for Delivery', 'Handed Over')),

    scope_summary TEXT NOT NULL DEFAULT '' CHECK (char_length(scope_summary) <=
3000),
    sales_owner_name TEXT NOT NULL DEFAULT 'Aarav Mehta (Sales Rep)',
    delivery_owner_id TEXT NULL,
    delivery_owner_name TEXT NULL,

    -- Embedded JSONB checklists with payload size sanity bounds
    deliverables JSONB NOT NULL DEFAULT '[]'::jsonb
        CHECK (jsonb_typeof(deliverables) = 'array' AND jsonb_array_length(deliverables) <= 30),
    dependencies JSONB NOT NULL DEFAULT '[]'::jsonb
        CHECK (jsonb_typeof(dependencies) = 'array' AND jsonb_array_length(dependencies) <=
30),
    document_links JSONB NOT NULL DEFAULT '[]'::jsonb
        CHECK (jsonb_typeof(document_links) = 'array' AND jsonb_array_length(document_links)
<= 20),
    activity_logs JSONB NOT NULL DEFAULT '[]'::jsonb
        CHECK (jsonb_typeof(activity_logs) = 'array' AND jsonb_array_length(activity_logs) <=
100),
```

```
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Index for fast dashboard status filtering and sorting
CREATE INDEX IF NOT EXISTS idx_handovers_status_updated
ON public.handovers (status, updated_at DESC);

-- Enable Row Level Security (RLS)
ALTER TABLE public.handovers ENABLE ROW LEVEL SECURITY;

-- Scoped Demo RLS Policies: Allow SELECT, INSERT, UPDATE for public demo, but FORBID DELETE
-- (Demo resets use deterministic UPSERT on fixed UUIDs so rows cannot be deleted maliciously)
DROP POLICY IF EXISTS "demo_select_handovers" ON public.handovers;
CREATE POLICY "demo_select_handovers" ON public.handovers FOR SELECT TO anon, authenticated USING (true);

DROP POLICY IF EXISTS "demo_insert_handovers" ON public.handovers;
CREATE POLICY "demo_insert_handovers" ON public.handovers FOR INSERT TO anon, authenticated WITH CHECK (true);

DROP POLICY IF EXISTS "demo_update_handovers" ON public.handovers;
CREATE POLICY "demo_update_handovers" ON public.handovers FOR UPDATE TO anon, authenticated USING (true) WITH CHECK (true);
```

# Deliverable G: Data-Access & API Contract (src/services/handoverService.js)

| Service Method | Underlying Operation | Role Guard | Request Arguments | Return Shape | Validation & Error Handling |
|---|---|---|---|---|---|
| initAndListHandovers() | SELECT * FROM handovers ORDER BY updated_at DESC (or localStorag | Any (sales, delivery, ops) | None | { mode: 'SUPABASE_CONNECTED' \| 'LOCAL_DEMO', handovers: | If Supabase table is empty, auto-seeds via upsert(INITIAL_HANDO |

| | | | | | |
|---|---|---|---|---|---|
| | e read) | | | Array, warning: string \| null } | VERS). If initial network request fails, switches to LOCAL_DEMO with a warning banner. |
| saveHandoverChanges(id, updates, logAction, actor) | UPDATE handovers SET ... WHERE id = :id | Any (unless status === 'Handed Over') | id, partial fields, logAction, actor | Updated Handover object | Rejects if status === 'Handed Over'. Re-evaluates readiness; auto-downgrades Ready for Delivery → Blocked if <6/6 gates pass. Appends activity log. |
| markReadyForDelivery(id, actor) | UPDATE handovers SET status = 'Ready for Delivery' | sales, ops | id, actor | Updated Handover object | Synchronously runs evaluateReadiness(). Throws error if passedCount < 6 or unauthorized role. |
| acceptHandover(id, actor) | UPDATE handovers SET status | delivery, ops | id, actor | Updated Handover object | Verifies current status is |

| | | | | | |
|---|---|---|---|---|---|
| | = 'Handed Over' | | | | Ready for Delivery and passedCount === 6. Locks handover as Handed Over. |
| returnToSales(id, actor, reason) | UPDATE handovers SET status = 'Blocked' | delivery, ops | id, actor, reason | Updated Handover object | Requires reason.trim().length >= 5. Sets status to Blocked and logs rejection reason. |
| resetDemoData() | UPSERT INITIAL_HANDOVERS ON CONFLICT (id) + localStorage reset | Any | None | Fresh INITIAL_HANDOVERS array | Restores all 4 demo projects to exact starting state without needing DELETE permission. |
| switchToLocalMode() | Locks module state to LOCAL_DEMO | Any | None | Current handovers synced to localStorage | Invoked explicitly by user if Wi-Fi drops mid-session during a write operation. |

# Deliverable H: Security, Persistence Fallback & Activity History Plan

1. **Secret Key Hygiene:** Only VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY (publishable key) are used in the frontend. **Never** place a Supabase service_role or sb_secret_ key in frontend code or .env files.
2. **Public Demo RLS Honesty:** Because the top-bar Persona Switcher does not require judge login credentials, all requests use the anon database role. Role permissions are enforced in the UI and handoverService.js, while the database enforces data integrity via CHECK constraints and blocks DELETE queries via scoped RLS.
3. **Unambiguous Two-State Persistence (No Split-Brain Merging):**
    - On app boot, the service resolves into either SUPABASE_CONNECTED or LOCAL_DEMO.
    - If a write fails while in SUPABASE_CONNECTED mode, the service **does not** silently write to localStorage. Instead, it surfaces an error toast with an explicit **"Switch to Offline Local Mode"** button so the user remains in full control of where data lives.
4. **Application-Level Append-Only Activity History:** Every save and transition prepends a structured entry ({ id, actor_name, actor_role, action, timestamp }) to activity_logs via a single service helper. In the UI, this is accurately labeled **"Handover Activity History"**.

# Deliverable I: Project Folder Structure

Plaintext

```
handoffhq/
├── .env.example              # VITE_SUPABASE_URL and VITE_SUPABASE_ANON_KEY template
├── .gitignore
├── index.html
├── package.json
├── vite.config.js            # Configured with @vitejs/plugin-react and @tailwindcss/vite
├── supabase/
│   └── schema.sql            # PostgreSQL table, constraints, index, and scoped RLS policies
└── src/
    ├── main.jsx              # React DOM root
    ├── App.jsx               # Global state (activeUser, handovers, storageMode, selectedId, toasts)
    ├── index.css             # Single line: @import "tailwindcss";
```

```
    ├── constants/
    │   ├── demoUsers.js          # DEMO_USERS (Aarav, Priya, Vikram) &
DELIVERY_OWNERS
    │   └── seedData.js           # INITIAL_HANDOVERS (4 fixed-UUID projects incl. "Acme
Website Redesign")
    ├── lib/
    │   ├── supabaseClient.js     # Safe Supabase client initializer
    │   └── readinessEngine.js    # Pure function evaluateReadiness(handover) & URL validator
    ├── services/
    │   └── handoverService.js    # Two-state CRUD, readiness re-evaluation, role guards,
activity logger
    └── components/
        ├── Navbar.jsx            # Brand bar, Storage Mode pill, Persona Switcher, Reset Demo
button
        ├── DashboardView.jsx     # 4 KPI cards, status filter pills, project cards with readiness
bars
        ├── HandoverWorkspace.jsx # Left column editor (Overview, Deliverables, Dependencies,
Docs, History)
        └── ReadinessPanel.jsx    # Right sticky panel (Readiness score, 6 Gate cards, transition
CTAs)
```

# Deliverable J: Ordered Implementation Roadmap & Emergency Cut Path

| Phase | Step & Deliverable | Est. Time | Completion Criteria (Definition of Done) |
|---|---|---|---|
| **Phase 1: Setup & Data (1.0 hr)** | **Step 1:** Scaffold Vite React JS app & configure Tailwind v4 (@tailwindcss/vite). | 20 min | npm run dev renders styled Tailwind utility classes cleanly. |
| | **Step 2:** Create demoUsers.js and seedData.js with fixed UUIDs. | 40 min | "Acme Website Redesign" is seeded with 3 passing gates and 3 failing gates (R-02, R-04, R-06). |

| | | | |
|---|---|---|---|
| **Phase 2: Core Engine & Service (1.75 hrs)** | **Step 3:** Build src/lib/readinessEngine.js with all 6 deterministic rules and URL validation. | 45 min | Unit/console check confirms "Acme Website Redesign" scores 3/6 (50%) and lists exact blocker strings. |
| | **Step 4:** Build supabaseClient.js and handoverService.js (LOCAL_DEMO + SUPABASE_CONNECTED). | 60 min | All CRUD and state transitions work end-to-end in LOCAL_DEMO mode before touching cloud config. |
| **Phase 3: UI & Demo Loop (3.25 hrs)** | **Step 5:** Build Navbar.jsx with Persona Switcher, Mode Badge, and Reset Demo Data button. | 45 min | Toggling persona updates active role badge; Reset restores seed state immediately. |
| | **Step 6:** Build DashboardView.jsx with 4 KPI cards, status filters, and project cards. | 60 min | Clicking "Acme Website Redesign" opens the workspace; KPI numbers match project states. |
| | **Step 7:** Build ReadinessPanel.jsx and HandoverWorkspace.jsx (including "✨ Insert Full Scope Summary" button and Activity History). | 90 min | **WORKING DEMO CHECKPOINT (Hour 6.0):** Complete 60-second "Acme Website Redesign" demo runs smoothly locally! |
| **Phase 4: Cloud &** | **Step 8:** Run supabase/schema.s | 45 min | Live .vercel.app URL loads in |

| | | | |
|---|---|---|---|
| **Deploy (1.5 hrs)** | ql in Supabase, add env vars, push to GitHub, and deploy on Vercel. | | SUPABASE_CONNECTED mode and persists transitions across page reloads. |
| | **Step 9:** Rehearse 60-second judge pitch 5 times and record backup screen capture. | 45 min | Pitch finishes in under 60 seconds with zero typing delays. |

- **Emergency Time-Crunch Cut Path:** If behind schedule at Hour 6, skip external Supabase cloud configuration and deploy directly to Vercel in **LOCAL_DEMO (localStorage) mode**. The entire Readiness Engine, role switching, activity history, and 1-click reset will function identically for judges with zero cloud latency.

# Deliverable K: Risks, Mitigations & Decision Log

## K.1 Risk Register

| Risk | Impact | Mitigation Built Into Blueprint |
|---|---|---|
| **Tailwind v4 vs v3 config mismatch breaks build** | High | Specified exact @tailwindcss/vite plugin setup (@import "tailwindcss";) with zero legacy PostCSS config files. |
| **Venue Wi-Fi drops or Supabase pauses/errors during judging** | High | Explicit LOCAL_DEMO mode + 1-click "Switch to Offline Local Mode" recovery button. |
| **Previous judge leaves "Acme Website Redesign" in Handed Over state** | High | Permanent "Reset Demo Data" button in top navbar performs idempotent UPSERT reset in <2s. |

| | | |
|---|---|---|
| **User deletes all mandatory dependencies to fake 100% readiness** | Medium | Rule R-04 explicitly requires mandatoryDependencies.length >= 1. |

## K.2 Final Decision Log

| ID | Decision | Rationale | Status |
|---|---|---|---|
| **D-01** | React 18 + Vite (JavaScript .jsx) + Tailwind CSS v4 (@tailwindcss/vite) | Fastest, zero-config setup matching beginner JS/React skills. | **Locked** |
| **D-02** | Single handovers PostgreSQL table with JSONB arrays + Scoped RLS (no DELETE) | Atomic reads/writes, zero multi-table join bugs, and safe public demo reset via fixed-UUID UPSERT. | **Locked** |
| **D-03** | Explicit Two-State Storage (SUPABASE_CONNECTED vs LOCAL_DEMO) | Eliminates silent split-brain overwrites and guarantees offline demo resilience. | **Locked** |
| **D-04** | Top-Bar Persona Switcher (Sales, Delivery, Ops) | Enables frictionless 60-second multi-role demonstration without login screens. | **Locked** |
| **D-05** | Deterministic 6-Gate Readiness Engine (R-01 to R-06) with Auto-Downgrade Guard | Predictable, explainable core differentiator that enforces handover quality at every transition. | **Locked** |

# Deliverable L: Self-Contained Coding-Agent Handoff Specification

**Copy-Paste Prompt for Coding Agent:** Everything below this line is self-contained and ready to hand directly to your coding agent to generate the complete application code.

## 1. Role & Objective

You are the implementation coding agent for **HandoffHQ**, a **Sales-to-Delivery Handover Command Center** for service agencies. Build a complete, polished, production-ready React + Vite + Tailwind CSS v4 web application that implements the deterministic 6-gate **Readiness Engine**, two-state persistence (SUPABASE_CONNECTED and LOCAL_DEMO), role-gated status transitions, append-only activity history, and the 60-second **"Acme Website Redesign"** judge demo workflow.

## 2. Tech Stack & Exact Package Setup

- **Language:** Plain JavaScript (.js / .jsx). Do NOT use TypeScript.
- **Dependencies:** react, react-dom, lucide-react, @supabase/supabase-js.
- **Dev Dependencies:** vite, @vitejs/plugin-react, tailwindcss, @tailwindcss/vite.
- **vite.config.js:**

JavaScript
```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

- **src/index.css:**

CSS
```
@import "tailwindcss";
```

- **Environment Variables:** Only read import.meta.env.VITE_SUPABASE_URL and

import.meta.env.VITE_SUPABASE_ANON_KEY. Never reference a service_role key.

## 3. Pre-Seeded Personas & Initial Handovers (src/constants/)

### src/constants/demoUsers.js

Export DEMO_USERS and DELIVERY_OWNERS:

- DEMO_USERS:
  1. { id: 'usr_aarav', name: 'Aarav Mehta', role: 'sales', title: 'Account Executive' }
  2. { id: 'usr_priya', name: 'Priya Nair', role: 'delivery', title: 'Delivery Lead' }
  3. { id: 'usr_vikram', name: 'Vikram Desai', role: 'ops', title: 'Head of Agency Ops' }
- DELIVERY_OWNERS:
  1. { id: 'usr_priya', name: 'Priya Nair (Delivery Lead)' }
  2. { id: 'usr_rohan', name: 'Rohan Kapoor (Senior Technical PM)' }

### src/constants/seedData.js

Export ACME_SAMPLE_SCOPE (a 155-character detailed scope string for 1-click demo insertion) and INITIAL_HANDOVERS with these 4 fixed-UUID records:

1. **id: '11111111-1111-4111-8111-111111111111' — "Acme Website Redesign"**
   - client_name: 'Acme Corp', deal_value: 45000, target_launch_date: '2026-12-15', status: 'Blocked', sales_owner_name: 'Aarav Mehta (Sales Rep)'
   - **Starts with 3/6 Gates Passed (R-01, R-03, R-05) and 3 Blockers (R-02, R-04, R-06):**
     - scope_summary: 'Redesign site' (13 chars — fails R-02)
     - deliverables:
       - { id: 'del-acme-1', title: 'Responsive Figma Design System (12 Core Templates)', due_date: '2026-11-20' }
       - { id: 'del-acme-2', title: 'Webflow CMS Migration, SEO Redirects & Cross-Browser QA', due_date: '2026-12-10' }
     - dependencies:
       - { id: 'dep-acme-1', title: 'Approved Brand Guidelines & Vector Logo Kit', is_mandatory: true, is_resolved: true }
       - { id: 'dep-acme-2', title: 'Production DNS & Webflow Workspace Admin Access', is_mandatory: true, is_resolved: false } (fails R-04)
     - document_links:
       - { id: 'doc-acme-1', label: 'Signed Master Services Agreement (SOW)', url: '[https://notion.so/acme-redesign-sow-2026](https://notion.so/acme-redesign-sow-2026)' }
     - delivery_owner_id: null, delivery_owner_name: null (fails R-06)
     - activity_logs: Initial entry showing deal created by Aarav Mehta.
2. **id: '22222222-2222-4222-8222-222222222222' — "FinPulse Mobile App MVP"**
   - client_name: 'FinPulse FinTech', deal_value: 82000, target_launch_date: '2027-01-30', status: 'Ready for Delivery', all 6/6 gates satisfied, assigned to Priya

Nair (Delivery Lead).

3. **id: '33333333-3333-4333-8333-333333333333' — "Nexus B2B SEO & Content Retainer"**
    - client_name: 'Nexus Cloud', deal_value: 18500, target_launch_date: '2026-11-15', status: 'Handed Over', all 6/6 gates satisfied, assigned to Priya Nair (Delivery Lead).
4. **id: '44444444-4444-4444-8444-444444444444' — "VoltEdge Shopify Plus Migration"**
    - client_name: 'VoltEdge Gear', deal_value: 34000, target_launch_date: '2026-12-28', status: 'Blocked', 2/6 gates satisfied.

## 4. Deterministic Readiness Engine (src/lib/readinessEngine.js)

Export isValidHttpUrl(value) and evaluateReadiness(handover). Evaluate the 6 gates (R-01 through R-06) exactly as defined in Deliverable A.2 above. Return:

JavaScript
{
  passedCount,
  totalCount: 6,
  percentage: Math.round((passedCount / 6) * 100),
  isReady: passedCount === 6,
  gates,     // Array of { id, title, passed, currentStatusText, whyItBlocks, actionNeeded }
  blockers   // gates.filter(g => !g.passed)
}

### 5. Two-State Persistence & Service Guards (`src/services/handoverService.js`)
- Support explicit `SUPABASE_CONNECTED` and `LOCAL_DEMO` modes.
- Enforce role permissions and synchronous `evaluateReadiness()` verification on `saveHandoverChanges`, `markReadyForDelivery`, `acceptHandover`, and `returnToSales`.
- Automatically downgrade `Ready for Delivery` → `Blocked` if any edit causes `passedCount < 6`.
- Prepend a timestamped activity log entry on every mutation.

### 6. UI Components & Visual Polish
- Design a clean, high-contrast B2B SaaS interface using slate/indigo/emerald/amber/rose Tailwind palettes.
- Include the **`"✨ Insert Full Scope Summary"`** quick-fill button next to the Scope Summary textarea when `scope_summary.trim().length < 30` so the presenter can resolve Gate `R-02` in one click during the 60-second demo.
- Display clear role helper hints in `ReadinessPanel.jsx` whenever the current persona cannot click a transition button (e.g., *"Switch role to Priya Nair (Delivery Lead) in the top bar to accept

this handover"*).
```eof

I have compiled the complete, amended **HandoffHQ Architecture & Implementation Blueprint** into `HandoffHQ_Final_Architecture_Blueprint.md` in the editor.

### Next Actionable Step
You can export or copy `HandoffHQ_Final_Architecture_Blueprint.md` directly, or reply **"Generate the application code"** and I will build the complete HandoffHQ web application for you right now.
```